



דו"ח פרויקט Shuttle Bus Vehicle & Driver Scheduling

קורס: "מערכות שירות"

מרצה: פרופ' טל רביב

מוסד אקדמי: אוניברסיטת תל אביב הפקולטה להנדסה ע"ש איבי ואלדר פליישמן המחלקה להנדסת תעשייה

סקירת גישת הפתרון

בפרויקט זה מימשנו פתרון היוריסטי לבעיית שיבוץ משולבת של רכבים ונהגים. מטרת המימוש הייתה לייצר פתרונות חוקיים ואיכותיים בזמן ריצה קצר, תוך שמירה על כלליות והימנעות מהתאמה ידנית למופעים הספציפיים שפורסמו. לכן, בשלב זה, פיתחנו גישה היוריסטית ולא במודל אופטימיזציה מדויק מלא, כגון MILP, שעלול להיות כבד מדי עבור מופעים גדולים יותר.

הSOLVER מקבל קובץ instance בפורמט JSON וקורא ממנו את כל הפרמטרים, זמני הנסיעה והנסיעות המתוכננות, ומייצר קובץ solution בפורמט הנדרש. הפלט מורכב מרשימת Duties כאשר כל duty מייצג משמרת אחת של נהג על רכב אחד, כולל רצף פעולות מלא: יציאה מהדיפו, נסיעות שירות, זמני המתנה, הפסקה, וחזרה לדיפו.

העיקרון המרכזי בגישה שלנו הוא בניית משמרות באמצעות שרשרת נסיעות שירות תואמות, במקום יצירת משמרת נפרדת לכל נסיעה. באופן זה ניתן לצמצם את מספר הנהגים, מספר הרכבים וזמני ה-deadhead. לאחר בניית המשמרות, מתבצע שלב נוסף של שימוש חוזר ברכבים פיזיים בין משמרות שאינן חופפות בזמן.

בחירות מידול וייצוג הפתרון

הSOLVER מייצג כל פתרון כרשימת Duties כאשר כל duty כולל:

- מזהה Duty
- מזהה נהג
- מזהה רכב
- זמן תחילת וסיום משמרת
- זמן ומיקום הפסקה
- רשימת פעולות רציפה בזמן ובמיקום

הפעולות האפשריות הן:

- Deadhead - נסיעה ריקה בין הדיפו לאחת התחנות או להפך.
- Service - נסיעת שירות מתוכננת מתוך הלו"ז.

- Wait - המתנה בתחנה או בדיפו.
- Break – הפסקת נהג.

המימוש שומר על פורמט JSON מינימלי ותואם לסכמת הפתרון. לא נוספו שדות עזר לקובץ הפלט, כדי למנוע כישלון בבדיקת schema או בבדיקת הפתרון הרשמית.

זמני הנסיעה מחושבים באמצעות מחלקת TravelTable אשר קוראת את טבלת זמני הנסיעה עבור כל נסיעה, בין אם מדובר בנסיעת שירות ובין אם ב-deadhead משך הנסיעה מחושב לפי זמן היציאה בפועל. כך נשמרת התאמה לזמני הנסיעה התלויים בשעה ביום.

אלגוריתם הפתרון

אלגוריתם הפתרון מורכב מכמה שלבים עיקריים:

1. **מיון הנסיעות** - בשלב הראשון כל נסיעות השירות ממוינות לפי זמן יציאה ולאחר מכן לפי מזהה נסיעה. המיון הכרונולוגי מאפשר לבנות duties בצורה טבעית לאורך יום הפעילות, ולהבטיח שהפתרון דטרמיניסטי וניתן לשחזור.
 2. **בניית duties באמצעות greedy chaining** - ה-SOLVER מתחיל מהנסיעה המוקדמת ביותר שטרם שובצה, ויוצר ממנה route ראשוני. לאחר מכן הוא מנסה להוסיף ל-route נסיעות נוספות שיכולות להתבצע על ידי אותו רכב ואותו נהג.
- בכל ניסיון הוספה נבדקים כמה תנאים:

- האם ניתן להגיע לנקודת המוצא של הנסיעה הבאה בזמן.
- האם רצף הנסיעות נשאר רציף בזמן ובמיקום.
- האם ניתן להכניס הפסקה חוקית בתוך המשמרת.
- האם ניתן לחזור לדיפו בסוף המשמרת.
- האם אורך המשמרת נשאר בטווח המותר.
- האם משך המשמרת אינו חורג מהפרמטר ההיוריסטי שנבחר לאותו ניסיון.

בשלב הראשון האלגוריתם מעדיף נסיעות שמתחילות בדיוק במקום שבו הסתיימה הנסיעה הקודמת. בחירה זו מפחיתה deadhead ושומרת על רצף תפעולי טבעי. אם אין מועמד מתאים, האלגוריתם מאפשר גם מעבר דרך הדיפו כדי להגיע לנסיעה בתחנה השנייה, אך רק כאשר הדבר עומד באילוצי הזמן והמשמרת.

3. **בחירת מועמד באמצעות מדיניות greedy** - האלגוריתם אינו בוחר בהכרח את כל השרשראים האפשריים. במקום זאת, הוא עובד בגישה גרידית עם מספר מדיניות. הפרמטר המרכזי הוא מגבלת אורך המשמרת



המועדפת בעת ניסיון להוסיף נסיעות ל-duty. הרצנו מספר ערכי מדיניות, למשל 510, 540, 570, 600, 630 ו-660 דקות, ובחרנו את הפתרון בעל העלות הנמוכה ביותר מבין הפתרונות החוקיים שהתקבלו.

גישה זו מאפשרת איזון בין שני שיקולים מנוגדים:

- משמרות קצרות יותר עשויות להפחית עלויות שעות נוספות.
- משמרות ארוכות יותר עשויות לאפשר כיסוי של יותר נסיעות באותו duty ולהפחית את מספר הנהגים והרכבים.

4. **הכנסת הפסקה** - עבור כל route מועמד, האלגוריתם מחפש חלון מתאים להפסקה. במקום להציב את ההפסקה באופן שרירותי, האלגוריתם בוחן את חלונות ההמתנה הקיימים לאורך המסלול:

- לאחר היציאה מהדיפו וההגעה לתחנת המוצא, לפני נסיעת השירות הראשונה.
- בין שתי נסיעות שירות, כאשר הרכב ממתין באחת התחנות.
- לאחר נסיעת השירות האחרונה ולפני החזרה לדיפו.

הפסקה נבחרת רק אם היא עומדת במגבלות הזמן ביחס לתחילת ולסיום המשמרת. בנוסף, היא חייבת להתבצע במקום שבו הרכב והנהג נמצאים בפועל באותו זמן. כאשר קיימים כמה חלונות אפשריים, ניתנת עדיפות לחלון המתנה גדול וטבעי, כדי להקטין את הצורך בפיצול משמרות או ביצירת נסיעות deadhead מיותרות.

5. **בניית רצף הפעולות** - לאחר שנמצא route חוקי, הפתרון בונה את רשימת הפעולות המלאה של Duty:

1. Deadhead מהדיפו לתחנת המוצא של הנסיעה הראשונה.
2. המתנה במידת הצורך.
3. נסיעות שירות לפי הסדר שנבחר.
4. המתנות בין נסיעות.
5. break במיקום ובזמן שנבחר.
6. Deadhead חזרה לדיפו בסוף המשמרת.

בכל שלב נשמרים שני תנאים: רציפות בזמן ורציפות במיקום. כלומר, כל פעולה מתחילה בדיוק בזמן שבו הפעולה הקודמת הסתיימה, והמיקום בסוף פעולה אחת מתאים למיקום בתחילת הפעולה הבאה.

6. **שיפור הפתרון באמצעות איחוד ושיבוץ מחדש של משמרות** - לאחר בניית הפתרון הראשוני, האלגוריתם מפעיל שלב שיפור. בשלב זה הוא מחפש duties עם מעט נסיעות שירות ומנסה לפזר את הנסיעות שלהם לתוך duties אחרים. עבור כל נסיעה, האלגוריתם מנסה להכניס אותה במיקומים שונים בתוך route קיים ולבדוק האם מתקבל duty חוקי.

אם כל הנסיעות של duty קטן מצליחות להשתלב בduties אחרים, ה duty המקורי נמחק. מטרתנו בשלב זה היא להפחית את מספר הנהגים ולעיתים גם את מספר הרכבים הדרושים. השלב מבוצע רק כאשר הוא שומר על חוקיות מלאה של הפתרון.

7. **שימוש חוזר ברכבים** - לאחר שנקבעו כל duties, מתבצע שיוך מזהי רכב. בשלב זה האלגוריתם ממין את המשמרות לפי זמן התחלה ומשתמש באלגוריתם greedy פשוט של לפי הלוגיקה הבאה:
- אם קיים רכב שסיים את משמרתו לפני תחילת ה- duty הנוכחי, נעשה בו שימוש חוזר.
 - אם אין רכב פנוי, מוקצה רכב חדש.

שלב זה מפחית את עלות הרכבים הקבועה מבלי לשנות את רצפי הפעולות או את שיבוץ הנהגים.

כיוול פרמטרים

הפרמטר המרכזי שכיילנו הוא max_append_shift_len כלומר אורך המשמרת המרבי המועדף בעת ניסיון לשרשר נסיעות לתוך duty. ניסינו ובדקנו מספר ערכים בטווח של 510 עד 660 דקות. עבור כל ערך נבנה פתרון מלא, ולאחר מכן חושבה העלות הכוללת. לבסוף נבחר הפתרון הזול ביותר.

הבחירה בטווח זה נועדה לאזן בין ניצול טוב של נהגים לבין הימנעות ממשמרות ארוכות מדי שיוצרות שעות נוספות. בנוסף, שמרנו על מגבלת זמן פנימית של כ-270 שניות, כדי להשאיר מרווח ביטחון ביחס למגבלת 5 הדקות של המטלה.

לא נעשה שימוש בפרמטרים מותאמים ידנית לפי מזהי נסיעות או לפי מופע ספציפי. כל הנתונים נקראים ישירות מהקבצים.

תוצאות חישוביות

הטבלה הבאה מציגה את התוצאות שהתקבלו על שלושת המופעים הנתונים שנבדקו. כל הפתרונות עברו את בדיקת התקינות המצורפת.

Instance	Feasible (Runtime in Seconds)	Duties	Vehicles	Deadhead minutes	Fixed vehicle cost	Variable cost	Driver cost	Total
small_1	Yes (8)	11	8	564	4,000.00	564.00	5,675.00	10,239.00
small_2	Yes (9.1)	13	8	564	4,000.00	564.00	6,306.25	10,870.25
medium_1	Yes (9)	19	11	807	5,500.00	807.00	9,250.00	15,557.00

השוואה ביחס לפתרון הייחוס

למופע small_01 פורסם פתרון ייחוס חוקי אך לא בהכרח אופטימלי. השוואה אליו מאפשרת להבין את איכות הפתרון שלנו.

Solution	Duties	Vehicles	Deadhead minutes	Driver cost	Total objective
Published reference	11	7	437	4,962.50	8,899.50
Our solver	11	8	564	5,675.00	10,239.00

פתרון הייחוס טוב יותר במופע זה. הפער נובע בעיקר משלושה גורמים:

1. שימוש ברכב אחד פחות.
2. פחות דקות deadhead.
3. עלות נהגים נמוכה יותר, כנראה עקב תזמון משמרות יעיל יותר.

עם זאת, האלגוריתם שלנו שומר על כלליות ואינו מכיל התאמות ידניות. מטרתנו הייתה לקבל פתרון חוקי ואמין עבור מספר מופעים, כולל מופעים גדולים יותר בעתיד, ולא לבצע אופטימיזציה ידנית למופע יחיד.

נקודות חוזק ומגבלות

היתרון המרכזי של הגישה הוא שהיא פשוטה, מהירה, וניתנת לשחזור. היא אינה דורשת ספריות חיצוניות או רישיונות מסויימים ולכן ניתן להריץ אותה בכל סביבת Python בסיסית. בנוסף, האלגוריתם מבוסס כולו על נתוני הקלט ואינו מניח מבנה ספציפי של לוח מסוים.

החיסרון המרכזי הוא שהאלגוריתם greedy ולכן אינו מבטיח אופטימליות. החלטה מוקדמת על שרשור נסיעות עשויה למנוע שרשור טוב יותר בהמשך היום. כמו כן, שלב הכנסת ההפסקות עשוי להוביל לעיתים לפיצול משמרות או להמתנות ארוכות יחסית בתחנות. ניתן לשפר זאת בעתיד באמצעות חיפוש שכונתי, החלפות בין duties או ניסוח set-partitioning שבו duties מועמדים נוצרים מראש ונבחרים על ידי אופטימיזציה.

סיכום

האלגוריתם שפותח מספק פתרון היוריסטי כללי לבעיית שיבוץ הרכבים והנהגים. הוא מצליח לייצר פתרונות חוקיים לשלושת המופעים הציבוריים שנבדקו, תוך עמידה בפורמט הפלט ובבדיקות החוקיות. האלגוריתם משלב שרשור greedy של נסיעות, הכנסת הפסקות בתוך חלונות המתנה, שיפור באמצעות ספיגת duties קטנים, ושימוש חוזר ברכבים בין משמרות לא חופפות.

למרות שהפתרון אינו אופטימלי, הוא מהווה baseline יציב ומהיר, ומתאים להמשך שיפור עבור מופעים גדולים יותר.



נספח שימוש ב-AI בכתיבת העבודה:

בעבודה זו נעשה שימוש בסיוע של כלי בינה מלאכותית לצורכי כתיבה, דיבוג ושיפור ההסבר הכתוב. הבחירות האלגוריתמיות הסופיות, המימוש, תהליך האימות והתוצאות שהוגשו נבדקו ואושרו על ידי צוות העבודה.